Data and Algorithm Analysis

Chapter 20 — Elementary Graph Algorithms

Thang Hoang

Department of Computer Science, Virginia Tech

Outline

Graphs

Breadth-First Search (BFS)

Depth-First Search (DFS)

Topological Sort

Table of Contents

Graphs

Breadth-First Search (BFS)

Depth-First Search (DFS)

Topological Sort

Undirected Graphs

An **undirected graph** $G = (V, E)$ is a tuple where V is a finite set of **vertices** and E is a set of 2-element subsets of V, called **edges.**
(Sometimes allow 1-element subsets, called loops.)

Example

$$\begin{aligned} V &= \{1, 2, 3, 4, 5, 6\} \\ E &= \{(1, 2), (1, 3), (1, 6), (2, 4), (2, 5), (3, 4), (3, 5)\}. \end{aligned}$$

Draw it!

Directed Graphs

An **directed graph** $G = (V, E)$ is a tuple where V is a finite set of **vertices** and $E \subseteq V \times V$ is a set of **edges** or **arcs.**

Example

$$\begin{aligned} V &= \{1, 2, 3, 4, 5, 6\} \\ E &= \{(1, 2), (1, 3), (1, 6), (2, 4), (2, 5), (3, 4), (3, 5)\}. \end{aligned}$$

Draw it!

Common Graph Representations

Assume $V = \{1, 2, \ldots, n\}$.

- **Adjacency list:** Array $A[1..n]$ where entry $A[i]$ is the list of vertices adjacent to vertex i
  Space complexity: $\Theta(|V| + |E|)$, good for sparse graphs
- **Adjacency matrix:** $n \times n$ matrix $M = (a_{ij})$, where

$$a_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E; \\ 0 & \text{otherwise.} \end{cases}$$

  Space complexity: $\Theta(|V|^2)$; good for dense graphs

Time complexity for individual operations: ?

General Graph Search Paradigm

Goal: Visit vertices of $G = (V, E)$ via adjacencies

- Color all vertices WHITE.
- Start at a source vertex s.
- Visit neighbors of current vertex in some order.
- Color vertex GRAY when found.
- Color vertex BLACK when all neighbors visited.
- If there are WHITE vertices not reachable from s, then choose another WHITE vertex as new source vertex.
- The edges of first discovery form a forest rooted at the source vertices.
- Works for both undirected and directed graphs.

Table of Contents

Graphs

Breadth-First Search (BFS)

Depth-First Search (DFS)

Topological Sort

Breadth-First Search (BFS)

Input: Graph $G = (V, E)$, either directed or undirected, and **source vertex** $s \in V$.

Output: $v.d =$ distance (smallest # of edges) from s to v, for all $v \in V$. $v.\pi$ is v's **predecessor** on a shortest path $s \rightsquigarrow v$.

Predecessor subgraph contains edges (u,v) such that $v.\pi = u$. The predecessor subgraph forms a tree, called the **breadth-first tree**.

Intuition — BFS

Expand the frontier between discovered and undiscovered vertices uniformly across the breath of the frontier.
Discover vertices in waves, starting from s.

- First visits all vertices 1 edge from s.
- From there, visits all vertices 2 edges from s, and so on.

BFS uses a first-in/first-out (**FIFO**) **queue** Q to hold the GRAY vertices.
Paths in BFS tree give shortest paths from s to any vertex v reachable from s.
Let $\delta(u, v)$ be the length of a shortest path from u to v or ∞ if no such path. Then $\delta(s, s) = 0$ and $\delta(s, v) = 1$ if $(s, v) \in E$.

Pseudocode — Breadth-First Search

```
BFS(G, s)
    for u ∈ V
        u.color = WHITE; u.d = ∞; u.π = NIL
    s.color = GRAY; s.d = 0
    Q = ∅
    ENQUEUE(Q, s)
    while Q ≠ ∅
        u = DEQUEUE(Q)
        for v adjacent to u
            if v.color == WHITE
                v.color = GRAY; v.d = u.d + 1; v.π = u
                ENQUEUE(Q, v)
        u.color = BLACK
```

Shortest-Path Tree

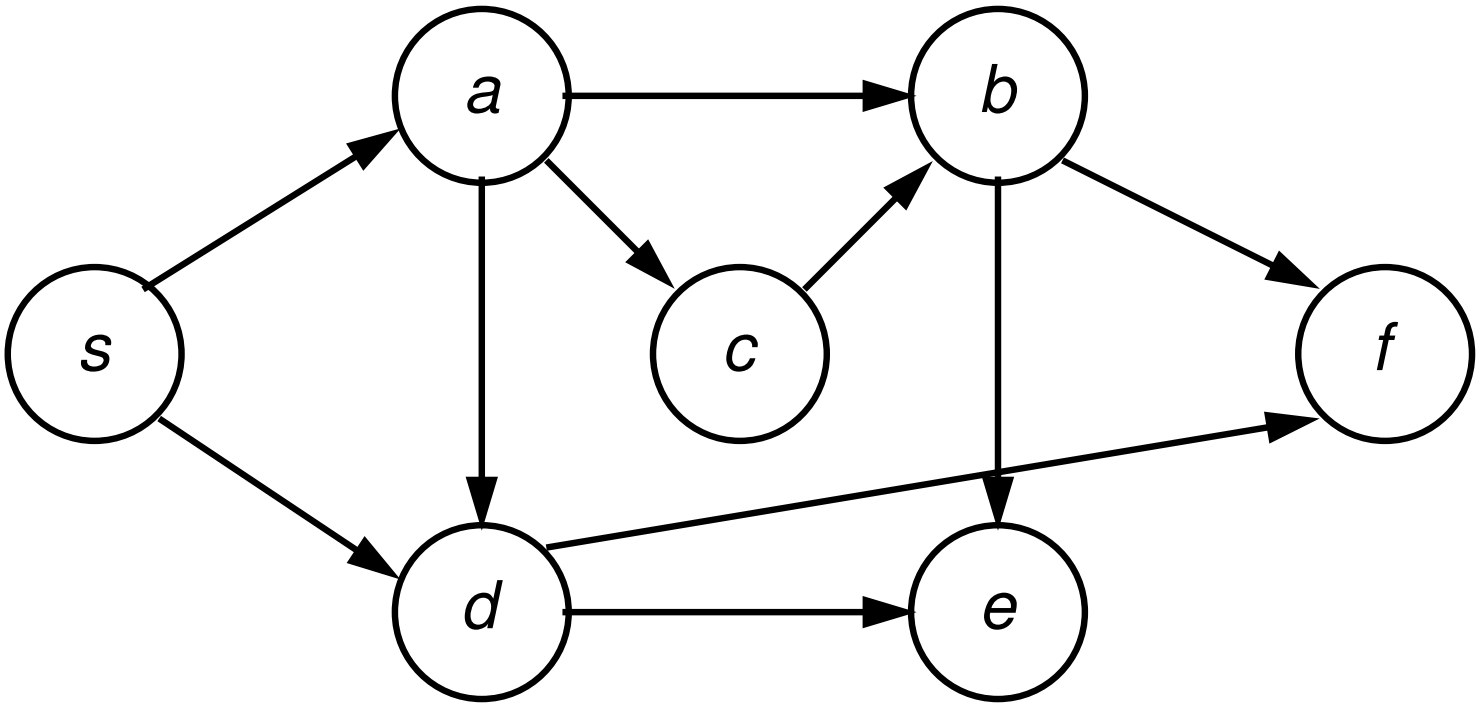
Theorem

For all $v \in V$, $\delta(s, v)$ equals the value $v.d$ assigned in line 10.

Proof.

By induction on $\delta(s, v)$. □

Example for BFS



- Vertex colors progress WHITE $\rightarrow$ GRAY $\rightarrow$ BLACK
- GRAY vertices are in Q
- Identify BFS tree from π pointers

BFS Time Complexity

Total Running Time: ?

Table of Contents

Graphs

Breadth-First Search (BFS)

Depth-First Search (DFS)

Topological Sort

Depth-First Search (DFS)

Input: $G = (V, E)$, directed or undirected (**No** source vertex given).

Output: 2 timestamps on each vertex:

- $v.d$ = discovery time
- $v.f$ = finish time

$v.\pi$ is v's predecessor in the depth-first forest of ≥ 1 depth-first trees. If $u = v.\pi$ then (u, v) is a **tree edge**.

Intuition — Depth-First Search

Methodically explores every edge.

- Start over from different vertices as necessary.

As soon as a vertex is discovered, explore from it.

- Use a **stack** to store exploring vertex (hence is recursive).

As DFS progresses, every vertex has a color:

- WHITE = undiscovered
- GRAY = discovered, but not finished exploring
- BLACK = finished (have found everything reachable from it)

Discovery and finish times:

- For all v, $1 \leq v.d < v.f \leq 2|V|$.

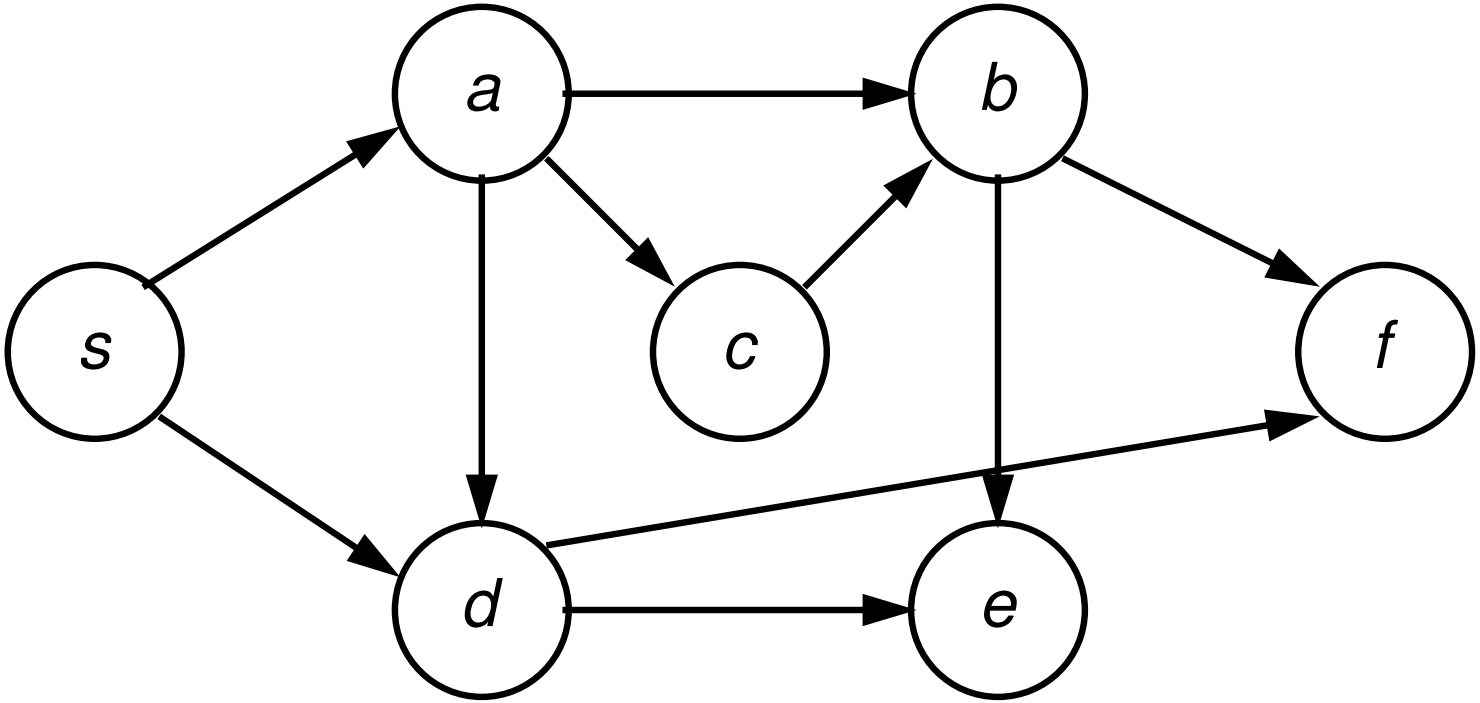
Pseudocode — Depth-First Search — Initialization

```
DFS(G)
    for u ∈ V
        u.color = WHITE
        u.π = NIL
    time = 0
    for u ∈ V
        if u.color == WHITE
            DFS-VISIT(G, u)
```

Pseudocode — Depth-First Search — Recursion

```
DFS-VISIT(G, u)
    time = time + 1
    u.d = time
    u.color = GRAY
    for v adjacent to u
        if v.color == WHITE
            v.π = u
            DFS-VISIT(G, v)
    u.color = BLACK
    time = time + 1
    u.f = time
```

Example for DFS



- Vertex colors progress WHITE $\rightarrow$ GRAY $\rightarrow$ BLACK
- GRAY vertices are on the stack
- Identify DFS forest from π pointers
- Pay attention to discovery and finishing times

DFS Time Complexity

Total Running Time: ?

Theorem 20.7

Theorem (Parenthesis theorem)

For all u, v, exactly one of the following holds:

1. $u.d < u.d < v.d < v.f$ *or* $v.d < v.f < u.d < u.f$ *(*$[u.d, u.f]$ *and* $[v.d, v.f]$ *are disjoint), and neither of u and v is a descendant of the other.*
2. $u.d < v.d < v.f < u.f$ *and v is a descendant of u*
   - *v is discovered after and finished before u.*
3. $v.d < u.d < u.f < v.f$ *and u is a descendant of v.*
   - *(u is discovered after and finished before v.*

$u.d < v.d < u.f < v.f$ (v is both discovered and finished after u) cannot happen (like parentheses)

- OK: ( ) [ ] ([ ]) [( )] | Not OK: ([ )] [( ])

White Path Theorem 20.9

Theorem

Let $G = (V, E)$ be an undirected or directed graph. In a depth-first forest of G, vertex v is a descendant of vertex u if and only if, at time u.d, there is a path of WHITE *vertices from u to v.*

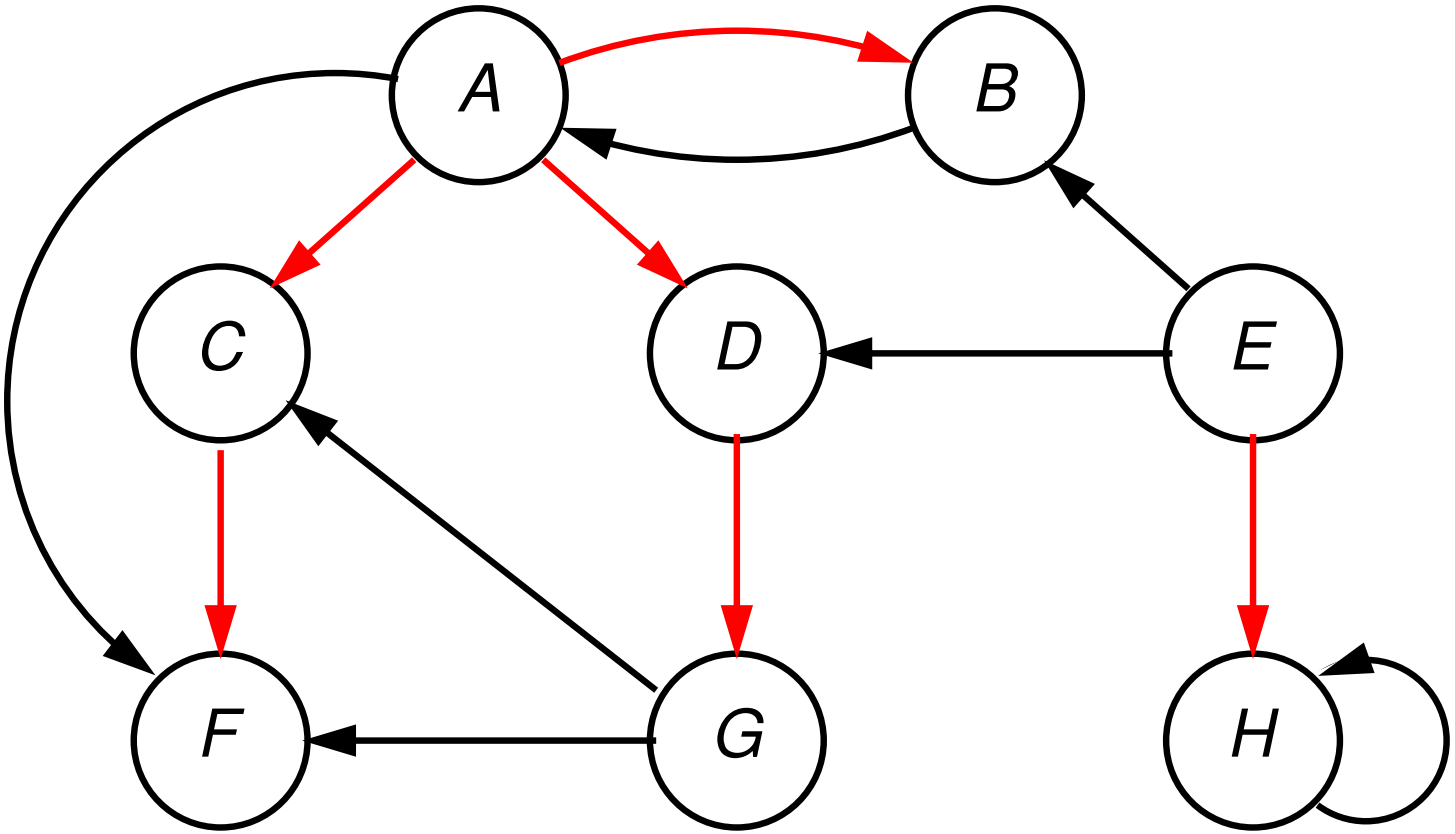
Proof.

By Corollary 20.8, if v is a descendant of u, then every vertex of the path in the forest from u to v is WHITE at time $u.d$. Conversely, assume

$$P = u, w_1, w_2, \ldots, w_k = v$$

is a path from u to v of all WHITE vertices at time $u.d$. Induct on k to show that all vertices on P are descendants of u. To the board!

Example — Depth-First Forest



Types of Edges

1. **Tree edges:** (u, v) is a **tree edge** if $v.\pi = u$.
   $\{(A, C), (C, F), (A, D), (D, G), (A, B), (E, H)\}$
2. **Back edges:** (u, v) is a **back edge** if it is not a tree edge and if v is an ancestor of u; includes loops.
   $\{(B, A), (H, H)\}$
3. **Forward edges:** (u, v) is a **forward edge** if (u, v) is not a tree edge and v is a descendant of u.
   $\{(A, F)\}$
4. **Cross edges:** All other edges are **cross edges.**
   $\{(G, C), (G, F), (E, B), (E, D)\}$

Theorem 20.10

Theorem

In a depth-first search of an undirected graph $G = (V, E)$, every edge is either a tree edge or a back edge.

Proof.

Let (u, v) be a non-tree edge such that $u.d < v.d$. By Theorem 20.9, since v is WHITE at time $u.d$, v is a descendant of u, u is an ancestor of v, and (u, v) is a back edge. □

Table of Contents

Graphs

Breadth-First Search (BFS)

Depth-First Search (DFS)

Topological Sort

Directed Acyclic Graph (DAG)

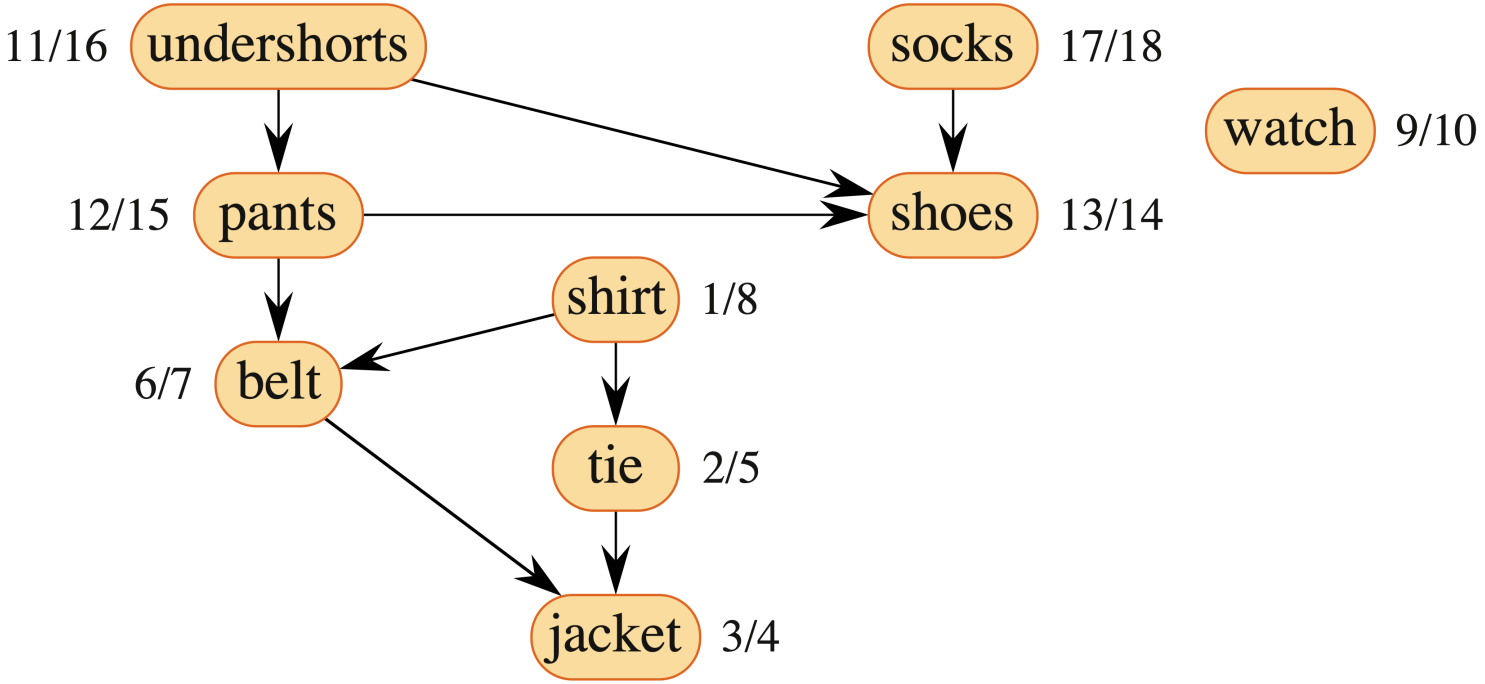
A directed graph with no cycles.

Good for modeling processes and structures that have a partial order:

- $a > b$ and $b > c \Rightarrow a > c$
- But may have a and b such that neither $a > b$ nor $b > c$.

Can always make a **total order** (either $a > b$ or $b > a \; \forall a \neq b$) from a partial order.
That's what a topological sort will do.

Example — DAG


11/16
undershorts
socks
17/18
watch
9/10
12/15
pants
shoes
13/14
shirt
1/8
6/7
belt
tie
2/5
jacket
3/4

Lemma 20.11

Lemma

A directed graph $G = (V, E)$ is acyclic if and only if a DFS of G yields no back edges.

Proof.

The existence of a back edge implies a directed cycle and that G is not acyclic. Now suppose G has a cycle

$$u, w_1, w_2, \ldots, w_k, u$$

with u discovered first. Then, the edge (w_k, u) is a back edge, since w_k is a descendant of u in the depth-first forest. □

Topological Sort

Topological sort of a DAG is a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v.

In a depth-first forest of G, sorting vertices into reverse order by finishing times gives a topological sort.

For example, the first vertex to finish has to have no outgoing edges (a sink) and can safely be put on the right.

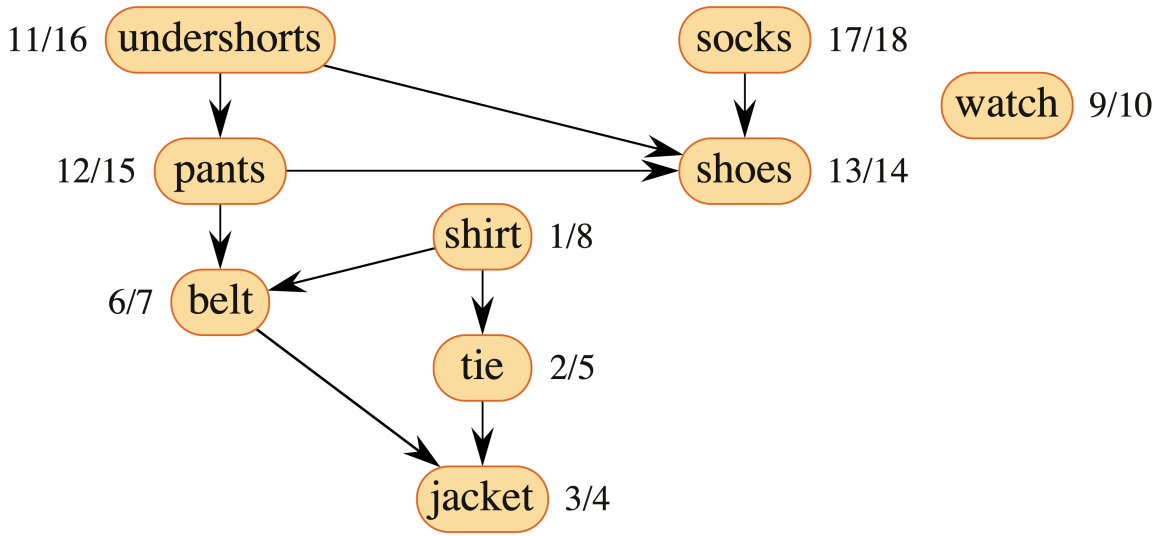
Pseudocode — Topological Sort

TOPOLOGICAL-SORT(G)
call DFS(G) to compute finish times $v.f \ \forall v \in G.V$
output vertices in order of decreasing finish times

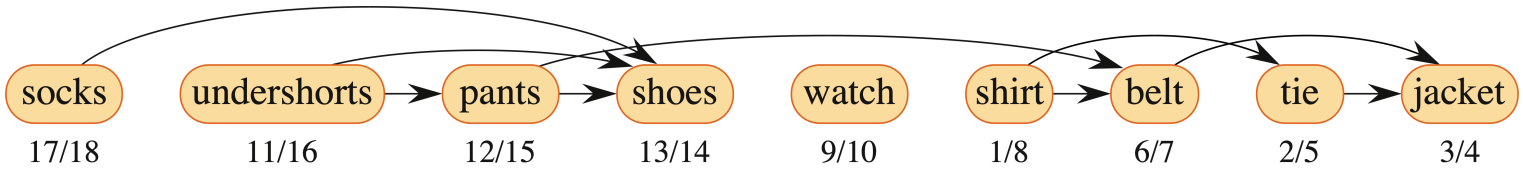
Time complexity: ?

Example — Topological Sort

DAG:



Topological Sort of DAG:



Theorem 20.12

Theorem

TOPOLOGICAL-SORT(G) *produces a topological sort of DAG G.*

Proof.

Let $(u, v) \in E$. It suffices to show that the DFS of G yields $v.f < u.f$.
At some time after $u.d$ but before $u.f$, edge (u, v) is examined.
If v is BLACK at that time, then $v.f < u.f$, as desired.
If v is WHITE at that time, then v is a descendant of u in the depth-first forest, so $u.d < v.d < v.f < u.f$, as desired.
We cannot have v being GRAY at that time, since then (u, v) is a back edge, and, by Lemma 22.11, G is not a DAG.
The theorem follows. □

Conclusions — Topological Sort

Topological sort can be solved in linear time using depth-first search.

Parting question: Can all topological sorts be obtained by a depth-first search?